Speaking of those integration tests, now that you've got the JavaScript up and running, let's see how the running JavaScript affects them.

Connecting the JavaScript to the Server Code

Were you to run the RSpec tests now that you've moved all the logic to JavaScript, you'd find that both tests in spec/system/add_task_spec.rb fail. The first test, which did not run in the JavaScript browser, fails because without JavaScript the tasks don't display at all. That's to be expected, and the test can be removed as its logic is now covered elsewhere.

The JavaScript test also fails for a couple of reasons. The simple one is the JavaScript makes a new Ajax call to get the JSON data for the tasks. To make that work, you need to make a small change to the ProjectsController#show method:

js_jasmine/03/app/controllers/projects_controller.rb

```
def show
  @project = Project.find(params[:id])
  respond_to do |format|
    format.html {}
    format.js { render json: @project.as_json(root: true, include: :tasks) }
  end
end
```

Here, the Rails respond_to method is being used to respond to a JavaScript request with the JSON representation of the task, adding project as a root element and adding in the tasks so as to match the expectations of the client side.

However, at this point the test still fails. Here's what the failing test looks like:

js_jasmine/02/spec/system/add_task_spec.rb

```
require "rails_helper"

RSpec.describe "adding a new task" do
  let!(:project) { create(:project, name: "Project Bluebook") }
  let!(:task_1) { create(
    :task, project: project, title: "Search Sky", size: 1, project_order: 1) }
  let!(:task_2) { create(
    :task, project: project, title: "Use Telescope", size: 1,
           project_order: 2) }
  let!(:task_3) { create(
    :task, project: project, title: "Take Notes", size: 1,
           project_order: 3) }

  it "can re-order a task", :js do
    visit(project_path(project))
    find("#task_3")
    within("#task_3") do
      click_on("Up")
    end
```